

Mixed-Precision Attention on Consumer Blackwell

A Pareto Replication Study from FP32 to NVFP4 on the RTX 5080

Five hand-rolled CUDA kernels, one controlled ablation.

Course project presentation
Spring 2026

[github.com/canoztas/
cmp674-attention-blackwell](https://github.com/canoztas/cmp674-attention-blackwell)

1. **The problem.** Why is attention slow? Why does it run out of memory?
2. **The hardware.** What is consumer Blackwell? What is the RTX 5080? Where do Tensor Cores fit in?
3. **The ladder.** Five hand-rolled kernels, V0 to V4, each changing one thing.
4. **The findings.**
 - ▶ Memory wall breaks at V2 ($17\times$ reduction).
 - ▶ V3 closes half the SDPA latency gap with FP8.
 - ▶ V4 **loses** to V3 by $\approx 30\%$ (the negative result).
5. **Why this matters.** What you should take away as a kernel developer or systems engineer.

The mechanism that made transformers possible

For each query q_i , look at every other token's key k_j , score the pair with $q_i \cdot k_j$, run a softmax over the scores, and use the resulting weights to mix the values v_j . Output is a weighted sum:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{D}}\right) V.$$

- ▶ Q, K, V are $N \times D$ matrices (sequence length N , head dimension D).
- ▶ $QK^\top$ is $N \times N$ – the score matrix.
- ▶ Softmax is row-wise; the result is the probability matrix P .
- ▶ PV gives the output O , which is again $N \times D$.

Why this is the bottleneck

The score matrix $S = QK^\top$ is **quadratic in N** . At sequence length 8,192 with batch 2 and 8 heads, that's $2 \cdot 8 \cdot 8192^2 \cdot \text{sizeof}(\text{float}) \approx 4.3$ GB just for S . At 16,384 it's 17 GB and the GPU runs out of memory.

Two problems, not one.

Problem 1: memory bandwidth

The big tensors (Q , K , V , S , P , O) live in HBM. Reading them once costs $O(N^2)$ bytes of HBM traffic per forward pass. On consumer Blackwell, HBM bandwidth is ≈ 960 GB/s; for $N = 8192$, head_dim 128, batch 2, heads 8, we move several GB through HBM *just* for S and P .

Problem 2: compute throughput

The two matmuls ($QK^\top$ and PV) dominate FLOPs. Tensor Cores accelerate them **only if** you feed them the right data layout and the right precision. FP32 ALU is $\sim 50\times$ slower than FP16 Tensor Cores; FP16 Tensor Cores are $\sim 2\times$ slower than FP8; FP8 are $\sim 2\times$ slower than FP4.

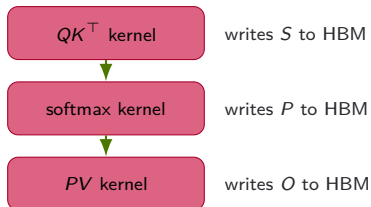
Two levers, then. Avoid materializing S and P (memory) *and* use the narrowest precision Tensor Cores you can (compute). Our paper measures both, in isolation.

Three separate kernel launches, all FP32:

1. QK^T **matmul** writes the $N \times N$ score matrix S to HBM.
2. **Row-wise softmax** reads S , writes the $N \times N$ probability matrix P back to HBM.
3. PV **matmul** reads P , writes the output O to HBM.

Numbers, $N = 2048$, **head_dim** = 128, **batch** = 2, **heads** = 8:

- ▶ HBM traffic per forward ≈ 1.0 GB
- ▶ FP32 ALU only – no Tensor Cores
- ▶ Latency $\approx$ **4,500** ms on RTX 5080
- ▶ Memory $\approx$ **640** MB



What this is good for

Correctness baseline. Every other variant is checked against this kernel's output. **V0 is our oracle.**

NVIDIA GeForce RTX 5080 (our card)

- ▶ Architecture: **Blackwell, sm_120a**
- ▶ **84** streaming multiprocessors
- ▶ **16 GB** GDDR7, ≈ 960 GB/s
- ▶ 5th-gen Tensor Cores
- ▶ Native FP8 (E4M3 / E5M2) and FP4 (NVFP4)
- ▶ Released early 2025

Datacenter Blackwell (B100/B200) for context

- ▶ Architecture: **sm_100a**
- ▶ Has **TMA** (Tensor Memory Accelerator)
- ▶ Has hardware-microscaled NVFP4 with row-major SMEM access
- ▶ CUTLASS 77_blackwell_fmha reference example works *only* on sm_100a / sm_103a

Consumer Blackwell looks like a smaller datacenter Blackwell, but **some of the most useful features are missing**. This matters for

Same instruction family (`mma.sync`), three precisions, three throughputs. The kernel asks for one explicitly via inline PTX or via the high-level `nvcuda::wmma` API.

Precision	Mnemonic	Width	Used by	Theoretical $\times$ FP32
FP16 (E5M10)	<code>HMMA.16816.F32</code>	$16 \times 8 \times 16$	V1, V2 (<code>wmma</code>)	1 $\times$
FP8 (E4M3)	<code>QMMA.16832.F32.E4M3.E4M3</code>	$16 \times 8 \times 32$	V3 (inline PTX)	2 $\times$
FP4 (E2M1)	<code>QMMA.16832.F32.E2M1.E2M1</code>	$16 \times 8 \times 32$	V4 (inline PTX)	4 $\times$

- ▶ **HMMA** = “Half MMA” (FP16 inputs, FP32 accumulator).
- ▶ **QMMA** = “Quarter MMA” on consumer Blackwell SASS; covers FP8 and FP4 with the precision encoded in the suffix.
- ▶ Each instruction uses a per-thread *fragment* layout fixed in PTX; you load operands into registers and call the instruction.
- ▶ FP4 specifically requires `sm_120a`, *not* plain `sm_120`.

FP8 (E4M3)

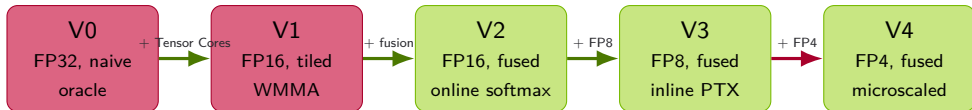
- ▶ 4 exponent bits, 3 mantissa bits
- ▶ Max representable magnitude: **448**
- ▶ ~ 16 representable positive values
- ▶ Reasonable for unscaled inputs around 1

FP4 (E2M1)

- ▶ 2 exponent bits, 1 mantissa bit
- ▶ Max representable magnitude: **6**
- ▶ ~ 6 representable positive values
- ▶ **Useless without per-block rescaling**

Microscaling, in plain language

The values you actually want to multiply (rows of Q , blocks of K , elements of P) live on a much wider scale than FP8 / FP4 can hold. **So you don't store the values directly.** You divide each block by its own absolute-max, store the small-range result, and multiply by that scale factor when you read it back. The mma instruction can operate on FP4 values; the scales live in FP32 and apply at the accumulator.



One change per step

The ladder is constructed so that each arrow changes *exactly one* design choice. The performance and accuracy delta at each step therefore quantifies the contribution of that one choice on this hardware, with no confounding.

Spoiler: V0 to V3 are wins. V3 to V4 is a **loss**.

- ▶ Three separate kernel launches; full S, P in HBM.
- ▶ **All FP32**: Tensor Cores not engaged at all.
- ▶ **Job**: pass the correctness test for every other variant. We check V1, V2, V3, V4 outputs against V0's output with relative-L2 tolerances.
- ▶ One **trap** we hit: the row-wise softmax reduction needs a `__syncthreads()` between the two reduction phases. Without it: nondeterministic $\sim 6\%$ output mismatch on large N . Took several hours to find.

Numbers ($N=2048, D=128$)

- ▶ Latency: **4,500** ms
- ▶ Memory: **640** MB
- ▶ Tensor Core insns: **0**
- ▶ Rel L2 vs FP32: **0** (it is the reference)

What this teaches

Naive attention is unworkable past tiny N .
Optimization is not optional.

- ▶ Same three-launch structure as V0, but inputs in FP16.
- ▶ Both matmuls go through the `nvcuda::wmma` API, which internally calls `HMMA.16816.F32`.
- ▶ **Trick #1:** get QK^T without an explicit transpose pass by loading K as `matrix_b col_major`. Same memory, transposed in operand-space.
- ▶ **Trick #2:** put all shared memory into a single dynamic allocation. Static + dynamic SMEM together count against a **48 KB** per-block default cap on Blackwell; we hit it exactly at $D = 128$.
- ▶ Still materializes S and P in HBM – this is V2's problem to solve.

Numbers ($N=8192$, $D=128$)

- ▶ Latency: **77.1 ms**
- ▶ Memory: **2176 MB**
- ▶ HMMA insns: **120 + 32**
- ▶ Rel L2 vs FP32: 6×10^{-4}

$\approx 60\times$ *faster than V0 already*, just from Tensor Cores. But still allocates 2 GB.

The big idea: **don't materialize S**

Process the keys and values in tiles, and combine the partial softmax results as you go. Maintain a running per-row **maximum** and a running **denominator**; rescale the partial output when a new maximum arrives.

$$\alpha^{(t)} = \exp(m^{(t-1)} - m^{(t)}), \quad O^{(t)} = \alpha^{(t)} O^{(t-1)} + \tilde{P}^{(t)} V^{(t)}.$$

- ▶ One fused kernel; one block per output row-tile.
- ▶ Per-block SMEM at $D = 128$ totals **96 KB**, over twice the default. Opt-in via `cudaFuncSetAttribute(..., MaxDynamicSharedMemorySize, 96K)`.
- ▶ Two boundary cases not in the FA paper but **required**: first iteration ($m = -\infty$),

The $17\times$ memory finding

At $N = 8192$, $D = 128$:

V1 = 2176 MB $\rightarrow$ V2 = 128 MB.

Same FP16 contract, no precision change.

The full $N \times N$ S, P never appear in HBM.

- ▶ Replaces V2's FP16 `wmma` with the FP8 (E4M3) `m16n8k32 sync mma`, called via **inline PTX**.
- ▶ Per-thread fragment layout is **fully specified** in the PTX ISA – not implementation-defined like in WMMA.
- ▶ Therefore we can keep the **output accumulator in registers** across the K/V loop and apply the per-iteration scaling in registers. V2 had to round-trip through SMEM.
- ▶ Per-tile FP8 scaling for Q , K , V , and P . One scale per tile, broadcast cooperatively, applied at the accumulator after each `mma`.
- ▶ Why not CUTLASS? Its FMHA reference example (`77_blackwell_fmha`) is gated to `sm_100a` / `sm_103a` only. **No FMHA reference for**

Numbers ($N=8192$, $D=128$)

- ▶ Latency: **25.8** ms ($2.4 \times$ V2)
- ▶ Memory: **128** MB (same as V2)
- ▶ QMMA insns: **96**
- ▶ Rel L2 vs FP32: 5×10^{-2}

Lever pulled

FP8 `mma` + register-resident accumulator. Both come from the same change: *the layout was knowable*.

- ▶ Same kernel structure as V3, but FP4 (E2M1) instead of FP8.
- ▶ Uses `kind::f8f6f4.m16n8k32`; FP4 packed into the *middle* 4 bits of each byte.
- ▶ Software microscaling: per-row, per-K-block (block size 32) FP32 scales, applied at the accumulator before each mma's accumulation.
- ▶ Each mma now does an extra ≈ 12 register ops to apply scales. Across 96 mmas per (kv iter, thread), this is ≈ 1100 ops/thread/iter **overhead**.
- ▶ Why not the **hardware** `mxvf4.m16n8k64`? Its operand layout is *scattered* per-thread, incompatible with our row-major SMEM idiom. Engaging it requires `ldmatrix`-style swizzled SMEM – multi-week scope.

Numbers ($N=8192$, $D=128$)

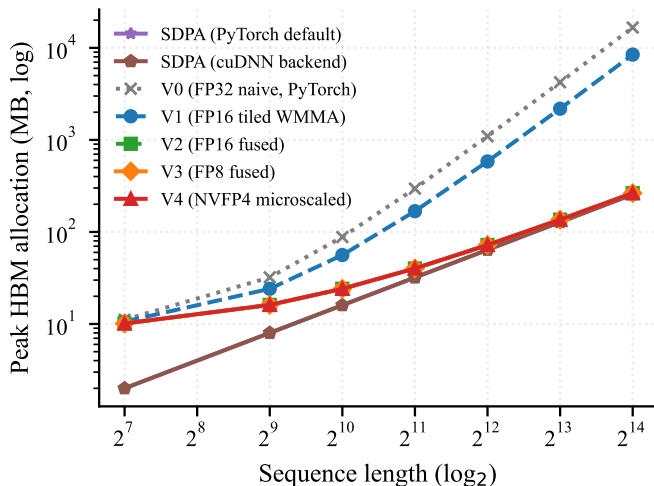
- ▶ Latency: **33.4** ms ($0.77\times$ V3, slower)
- ▶ Memory: **128** MB
- ▶ QMMA insns: **96**
- ▶ Rel L2 vs FP32: 0.21

Lever pulled *against* us

Software microscaling overhead $>$ FP4 throughput advantage. **This is a finding, not a failure.**

Finding 1 – Memory wall breaks at V2

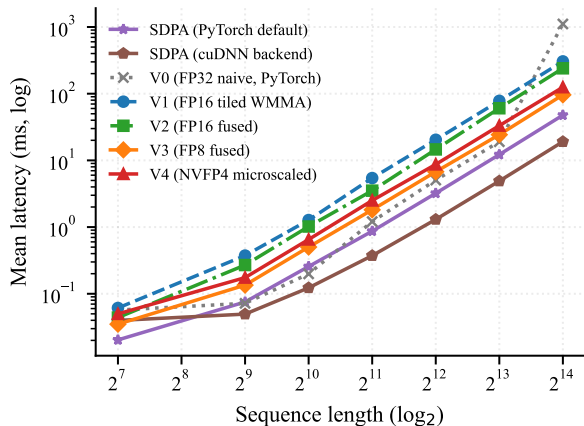
15 / 24



► V1 grows quadratically; V2 grows linearly.

Finding 2 – V3 closes half the SDPA gap with FP8

16 / 24



At $N = 8192$, $D = 128$:

- ▶ V2: ~ 60 ms
- ▶ V3: ~ 26 ms ($2.4\times$ V2)
- ▶ SDPA: ~ 12 ms

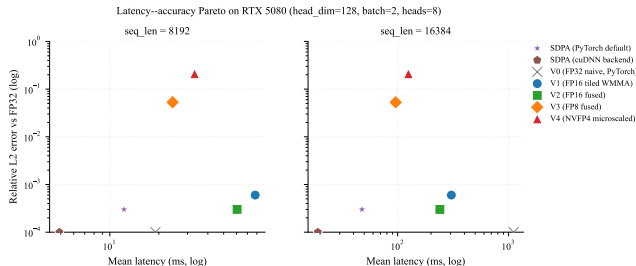
Why V3 wins so much

- ▶ FP8 mma throughput ($\sim 2\times$ FP16)
- ▶ Register-resident accumulator (V2 round-trips through SMEM)

Both come from the same change (PTX-specified fragment layout).

Finding 3 – V4 is **slower** than V3

17 / 24



- ▶ V4 (red triangle) sits at **higher latency** *and* **higher error** than V3 (orange diamond). Dominated by V3.
- ▶ **The negative result.** On consumer Blackwell with our software-microscaled path, FP4's hardware throughput advantage (theoretical $2\times$ FP8) is more than canceled by the per-mma overhead.
- ▶ The hardware-microscaled `mxsf4nvf4.m16n8k64` *would* plausibly invert this. Its scattered-fragment layout is the blocker – not the FP4 idea itself.

What we expected

PyTorch SDPA on the RTX 5080 dispatches to **FlashAttention-2 in FP16**. Beating it would mean beating a heavily-tuned FA-2 kernel.

What actually happens, in mid-2026, on Windows

The PyTorch 2.9.1+cu128 wheel was compiled *without* flash attention. cuDNN attention and memory-efficient attention are also runtime-disabled for our (sm_120, FP16, non-causal) inputs. SDPA **falls through to MATH backend** – a pair of cuBLAS matmuls with a softmax in between.

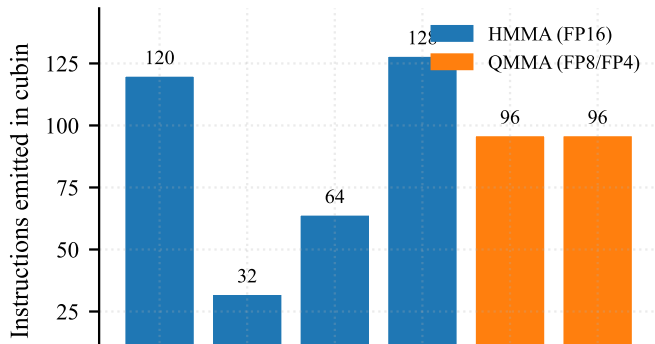
Concretely, when we tried to pin the dispatch:

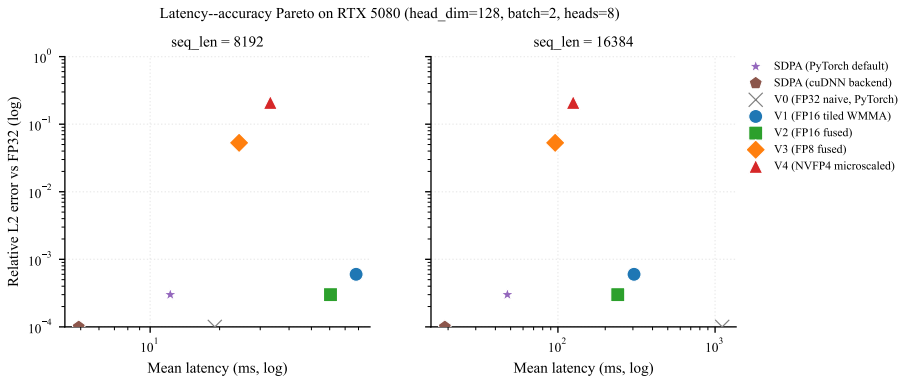
```
with torch.nn.attention.sdpa_kernel(SDPBackend.FLASH_ATTENTION):  
    F.scaled_dot_product_attention(Q, K, V) # raises NoBackendError
```

Methodological consequence: comparing a custom kernel “against SDPA” without naming the dispatch can be misleading. Always pin the backend and disclose it.

Nsight Compute's hardware counters require admin (ERR_NVGPUCTRPERM). On a consumer card without admin we cannot use it. **So we read the SASS directly.**

```
cuobjdump --dump-sass kernels/v3_flash_fp8/_C.cp311-win_amd64.pyd '  
| Select-String QMMA | Measure-Object | % Count  
# 96
```





- ▶ **V3 is the practical sweet spot** when you can absorb $\sim 5\%$ relative L2 error.
- ▶ SDPA is faster but has a hidden cost (it materializes S and P ; without flash, this is real HBM traffic).
- ▶ V4 sits dominated by V3 in this software stack.

1. **Algorithmic fusion is the biggest single win.** The $17\times$ memory reduction at $V1 \rightarrow V2$ came from changing the structure, not the precision.
2. **Narrower precision pays off only when the layout is knowable.** The $V2 \rightarrow V3$ latency win is half FP8 mma throughput, half register-resident O. Both require the fragment layout to be specified.
3. **Lower precision is not strictly better.** FP4 lost on consumer Blackwell because the cheap-microscaling path is incompatible with the layouts we use elsewhere. Hardware quirks defeat “obvious” optimizations.
4. **Vendor baselines are moving targets.** Pin your SDPA backend. State the build. Disclose the dispatch.
5. **Read the SASS.** `cuobjdump` tells you exactly which Tensor-Core instructions you emitted. That is more reliable than runtime metrics gated by admin permissions.

- ▶ **Hardware-microscaled NVFP4** (`mx4nvp4.m16n8k64`). Would likely flip V4 vs V3. Requires a swizzled SMEM layout incompatible with V3's idiom; multi-week scope on top of the existing kernel.
- ▶ **Backward pass.** Non-trivial for narrow-precision variants. The Attn-QAT paper documents specific instabilities; we focus on forward inference only.
- ▶ **RTX 5090 replication.** Same compute capability; very different SM count, memory bus, VRAM. Would tell us how much of the V4 result is hardware-microscaling-availability vs 5080-specific.
- ▶ **Backend pinning that actually works.** The SDPA FLASH_ATTENTION pin failed; on a Linux build with a flash-enabled wheel it would not. Re-running the comparison on Linux is a one-day item.

```
# Build the kernels (once per machine):  
. .\env\activate_for_build.ps1  
pip install --no-build-isolation -e kernels/v0_naive_fp32  
pip install --no-build-isolation -e kernels/v1_tiled_fp16  
pip install --no-build-isolation -e kernels/v2_flash_fp16  
pip install --no-build-isolation -e kernels/v3_flash_fp8  
pip install --no-build-isolation -e kernels/v4_flash_nvfp4  
  
# Run the full sweep + figures:  
python bench/run_all.py --config bench/configs/sweep_final.yaml '  
    --output bench/results/sweep_final.parquet  
python analysis/paper_figures.py
```

What you get

- ▶ One Parquet file with every per-iteration latency + provenance.
- ▶ Six figures and a LaTeX results table, regenerated from the Parquet.
- ▶ 495 passing pytest correctness tests across V0–V4.

Questions?

Code, paper, and slides:

`github.com/canoztas/cmp674-attention-blackwell`

CMP 674 – Parallel Computing with GPUs – Hacettepe University